

Table 1

Digits	Upper bound	Lower bound
1	1209	697
2	1336	697
3	1477	697
4	1209	770
5	1336	770
6	1477	770
7	1209	852
8	1336	852
9	1477	852
*	1209	941
0	1336	941
#	1477	941

For example, when you press the button '1' on a phone keypad, a signal consisting of a sine wave at 697Hz plus one at 1209Hz is generated. To fiddle

around with DTMF and its encoding and decoding, we have written a library. The library has the functions `dtmf_encoder()` to convert the input number to its corresponding DTMF audio, and `decode_dtmf()` to decode the same.

Okay, now let us try to encode a number like 76589410. We simply write the command:

```
tone_2 = dtmf_encoder('76589410')
IPython.display.Audio(tone_2, rate = Fs)
```

In the above command, we are converting each digit into DTMF audio signals by taking sine waves at the required frequencies.

For convenience, plotting the graph of such audio signals taking the sampling rate $F_s = 24000\text{Hz}$ is shown in Figure 1.

It is clearly visible from the plot that each number is a superposition of two different sine waves at two different frequencies.

Now let's encode a number and special character series of 1498*0:

```
tone_1 = dtmf_encoder('1498*0')
IPython.display.Audio(tone_1, rate = Fs)
```

The above command will also display the audio representations of the frequencies with a customisable speed. We can see their size and represent this in a graphical form, as shown in Figure 2.

Similarly, we may also decode the DTMF frequencies into required digits as before using a simple algorithm where we try to dissect the numbers according to the frequency ranges they represent. We check the upper bound and lower bound of the frequencies by converting it into NumPy arrays. For example, to decode '1498' we get a frequency between 697Hz to 1209Hz, so we know that the first digit is one. The next one lies between 770Hz to 1209Hz, and so on.

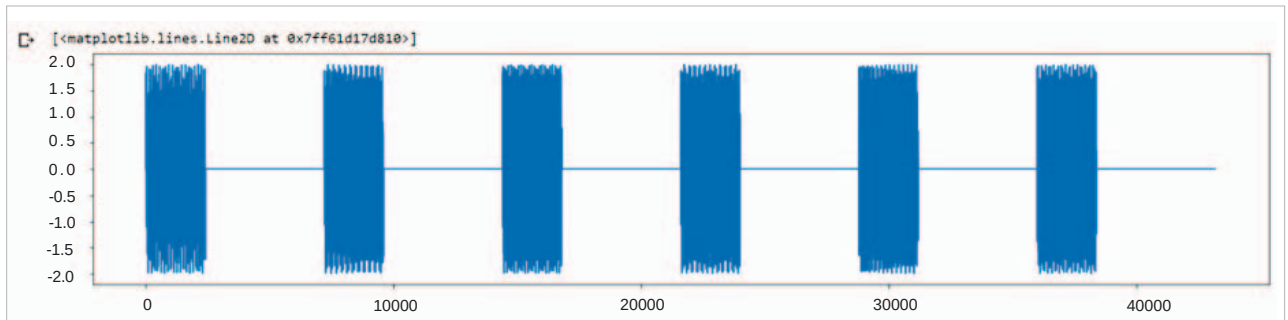


Figure 1: Audio signals graph

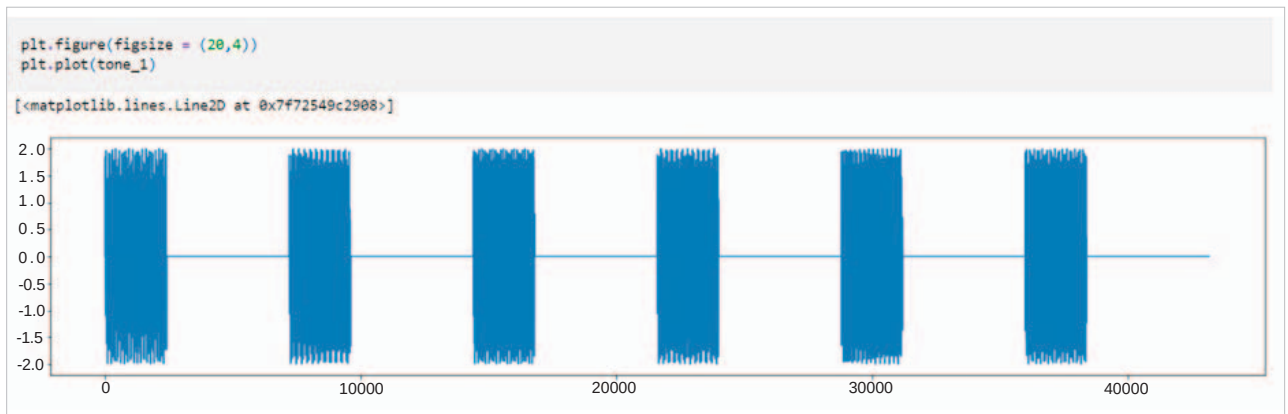


Figure 2: Sound frequencies with a customisable speed